Silicon & Handset — Executive Business & Macroeconomic Realism Audit

Perspective: Semiconductor/Hardware Chief Executive & Industrial Organization / Monetary Economist
Subject: Critical audit of economic rationality, unit economics, market microstructure, capital allocation, and industrial realism across `SPINOFF_SOURCE` (`v0.2.0` actual code, `v0.3.0 ACTIVE_PROMPT.md` specification, and Master Design Blueprints)
Target Audience: Project Owner & Implementing AI Engineers (`Codex`, `Agy`, `Gemini`)
Constraint Alignment: Fully compliant with `AGENTS.md` (immutable `human-core.js`, strict integer satang conservation, observer-only autonomy, zero rubber-banding).

Executive Verdict: How Realistic Is the Game Today?

Where Silicon & Handset Already Beats 95% of Business Simulations

From both an economist's and a hardware operator's standpoint, the **foundational philosophy** of *Silicon & Handset* is unusually rigorous:

1. **Strict Stock-Flow Accounting (`audit()`):** Unlike standard tycoon games that conjure money out of thin air when a product "sells" to an infinite void, your game enforces closed-system integer satang conservation and physical material conservation across raw inputs, transit, WIP, prototypes, and finished goods.
2. **Genuine Supply-Side Friction:** Time lags in transit (`12–72h`), prototype consumption, failed validation respins, finite household wallets, and shared labor budgets mean firms cannot scale instantaneously just by having cash.
3. **Zero Artificial Rubber-Banding:** Competitors succeed or fail based on their own balance sheets, teams, and choices—a prerequisite for any credible economic laboratory.

Where the Simulation Currently Breaks Economic & Commercial Logic

Despite strong accounting plumbing, inspecting the actual equations and parameters in `SPINOFF_SOURCE/js/sim.js`, `SPINOFF_SOURCE/js/phone-engineering.js`, and the Master Blueprints reveals **10 major economic and commercial distortions**. Left uncorrected, these flaws produce irrational firm behavior, artificial monopolies, zombie companies, broken Make-or-Buy incentives, and inevitable macroeconomic deflationary collapse.

Part I — Ten Things That Do Not Make Sense (Source-Quantified Diagnosis)

1. The Macroeconomic Deflationary Black Hole (Broken Circular Flow of Income)

- **What the code does (`sim.js`):** Total world money is fixed (`openingCapital`), which is sound *only if money circulates*. Instead:
  - Consumer cohorts (`value`, `photo`, `performance`) start with `1,200,000,000` satang (**THB 12,000,000**) total and **never receive income**.
  - `w.treasury` collects `1,200` satang per raw material unit, `4x` R&D lab fees, `THB 800–3,200` product NRE fees, and `THB 60/day` per person in meals (`~THB 1.86M/year` across ~85 people)—and **hoards 100% of it forever**.
  - Employees (`w.people`) collect daily wages and **never spend a single satang on phones**; their unspent wages sit idle in `p.cash` forever.
  - Worse, `w.treasury.stock` starts at `100,000` raw units and **never regenerates**.
- **Why an Economist Flags This:** By Fisher's Equation of Exchange ($MV = PQ$), fixing M while letting velocity $V \to 0$ (via two massive one-way liquidity sinks: `treasury.cash` and `people[].cash`) guarantees **systemic demand starvation**. By Year 2–3, consumers literally run out of money even if firms build great phones, while millions of baht sit sterilized inside `treasury` and employee mattresses.

2. Winner-Take-All Demand Cliff & Alphabetical ID Monopoly (`settleMarket()`)

- **What the code does (`sim.js`):**

```JavaScript
candidates.sort((a,b) => b.utility - a.utility || a.d.id.localeCompare(b.d.id));
const { d, utility } = candidates[0];
```

- **Why an Economist & Executive Flag This:**
  - Every buyer inside a segment uses the **exact same deterministic utility formula** and buys `candidates[0]` until that single model runs out of inventory or `salesWork`.
  - If Brand A scores `0.6501` utility and Brand B scores `0.6500` (a `0.015%` difference), Brand A captures **100% of demand** while Brand B gets **0% market share** until Brand A stocks out.

- Worse: when two brands produce identical specs and prices (common in early eras), `a.d.id.localeCompare(b.d.id)` awards 100% of sales to design `'d1'` over `'d2'` purely by alphabetical ID string order!
- **Real-world economics:** Differentiated consumer markets exhibit **horizontal taste heterogeneity** (modeled via Multinomial Logit discrete choice). Close competitors split market share smoothly; a 1-baht price cut does not flip market share from 0% to 100%.

3. Severe Scale Mismatch Across Wages, ASPs, BOMs, and Custom Silicon NRE

- **What the code & blueprints do:**
  - **Wages (`sim.js`):** Every staff member—from a junior technician to a Chief Silicon Architect or CEO—is spawned with `wage = 14,000..21,000` satang (**THB 140–210/day**, or **~THB 4,200–6,300/month**), below unskilled minimum wage, with **zero wage differentiation between CEO and engineer**.
  - **Unemployed Starvation:** Unemployed specialists (`outside-0..7`) start with `THB 1,000` (`100,000` satang) and pay `THB 60/day` for meals. On **Day 17**, every unemployed engineer in the world goes broke, stops eating meals permanently, yet still refuses job offers unless offered a premium over their old wage!
  - **Compressed ASP Ceilings (`SEGMENTS`):** The budget ceiling is **THB 1,900**, midrange camera is **THB 3,300**, and top-tier flagship (`performance`) is capped at **THB 5,200**! Yet at `level = 3`, raw component BOM + overhead in `estimatedCost()` already climbs to **~THB 3,000–3,700+**, compressing gross margins precisely where real smartphone brands earn their highest gross margins (flagship ASPs in reality are 8x–15x feature-phone ASPs, not 2.7x).
  - **Blueprint vs. Runtime NRE Contradiction:** The Master Blueprint cites real-world `$80M NRE` and `727,273` unit break-even volumes, while `phone-engineering.js` sets `maskCostSatang` around `THB 12,000–60,000` (recoverable in just a few dozen phones!), and the actual simulated world sells only **~5,000 phones/year across all firms combined**.
- **Why a Business Strategist Flags This:** Make-or-Buy is the soul of your game ($Q_{\text{breakeven}} = \frac{F_{\text{NRE}}}{P_{\text{buy}} - V_{\text{make}}}$). If F_{NRE} is too cheap (30 phones), **making your own chip is always dominant** (no dilemma). If F_{NRE} uses unscaled real-world millions in a 5,000-phone world, **making your own chip is 100% guaranteed suicide** (also no dilemma). NRE, wages, BOM, and cohort demand **must be dimensionally calibrated to the simulated market size**.

4. The Sunk-Cost Pricing Fallacy (`reviewStrategy()`)

- **What the code does (`sim.js`):**

```javascript
const floor = Math.round(cost * 1.08);
d.price = Math.max(floor, Math.min(target.maxPrice, Math.round(cost * margin * clearance)));
```

- **Why a Business Executive Flags This:**
  - First, firms use **pure Cost-Plus Pricing**, ignoring competitor prices and market elasticity—if a firm has high costs due to bad procurement, it *raises* its price, pricing itself out of the market!
  - Second, `floor = cost * 1.08` enforces a **hard refusal to ever sell below historical estimated cost + 8%**, even when `d.inventory > 18` is obsolete dead stock and the company is running out of cash to pay wages!
  - **First rule of corporate finance & microeconomics:** Manufacturing cost of already-built inventory is a **sunk cost**. When technology advances or liquidity runs low, rational firms mark down obsolete stock below BOM cost (clearance liquidation) to convert dead inventory back into working capital.

5. Blind Supplier Production & Frozen Supplier Pricing (`supplierQuotes` & `operate`)

- **What the code does (`sim.js`):**
  - Each supplier's `priceBps` (`80%–125%`) and `leadHours` (`12–72h`) are **rolled once at `create()` and never change**. A high-priced supplier losing 100% of orders to its twin competitor never lowers prices; a supplier facing surging demand only gets a binary `+12%` bump when `stock < 12`.
  - Worse, in `operate()`, suppliers loop `for (let l = 0; l <= f.capability; l++)` and produce whichever generation has the lowest stock below `120` units! That means an advanced supplier keeps burning scarce cash and raw materials to maintain **120 units of obsolete `level 0` feature-phone parts** even after every handset brand in the economy has moved to `level 2–3`!
- **Why an Industrial Economist Flags This:** Suppliers in reality are profit-seeking firms that produce **to downstream demand signals / order backlogs** and adjust **bid/ask quotes dynamically** based on inventory cover and capacity utilization.

6. Hardcoded EMS Monopoly & Zero Economies of Scale

- **What the code does (`sim.js`):**
  - Line 66 hardcodes `const ems = firm(w, 'supplier-ems')` for assembly line fees (`THB 240`), completely ignoring `supplier-ems-alt` (even if `player` chose to start as an EMS firm, brands wouldn't pay `player` the assembly fee!).

    - Every handset production run is locked to `batch = 12` units, and every supplier batch is locked to `quantity = 12` units, with **constant unit costs**.
- **Why an Operations Executive Flags This:** Semiconductor and electronics industries are defined by **Economies of Scale (EOS)** and **Minimum Order Quantities (MOQ)**. Without volume cost curves (fixed setup/tooling amortization per batch + variable unit cost), a giant brand ordering 500 phones has zero unit-cost advantage over a micro-brand ordering 12 phones, and small niche brands face no MOQ trade-off.

7. Immortal "Zombie Firms" & Zero Insolvency / Credit Mechanics

- **What the code does (`daily()`):** When a company cannot pay daily wages (`f.cash < due`), `f.paidToday = false`, employees increment `unpaidDays++`, stop working, and **stay employed at the bankrupt firm forever** (unless slowly poached one by one, leaving at least 3 staff trapped forever because `recruit()` checks `firm.people.length > 3`!). Meanwhile, the bankrupt firm holds onto unsold phones and raw parts forever without liquidating them.
- **Why an Economist Flags This:** Creative destruction requires **exit and capital reallocation**. Insolvent firms that miss payroll repeatedly must first attempt **emergency inventory liquidation**, and upon prolonged insolvency, release unpaid staff into the labor pool and auction remaining physical stock rather than trapping scarce labor and materials in frozen zombie entities.

8. Misallocated Sales Labor & Midnight Pipeline Amnesia

- **What the code does (`tick()` & `daily()`):**
    - Every working employee at a brand—even deep-tech R&D engineers—automatically diverts **15% of every hour** into `salesWork`.
    - At `hour === 0`, `f.salesWork = 0` wipes out 100% of unused sales effort! If a firm spends 60 days building up to launch, 100% of the `15%` sales labor spent during those 60 pre-launch days is **completely wasted** because no product was `released` yet, and on launch day it can only sell as many phones as a single day's `salesWork / 25` allows!
- **Why a Commercial Executive Flags This:** Pre-launch commercial work builds **channel readiness / order pipeline**, and post-launch sales momentum decays gradually (e.g., `15%–25%` daily decay, capped by stock/pipeline capacity), not instantaneous midnight amnesia. Moreover, commercial output should weight staff with actual commercial/sales roles higher rather than taxing pure silicon architects equally during R&D phases.

9. Treating Durable Goods Like Perishable Groceries (No Installed Base Saturation)

- **What the code does:** Consumers buy `5 / 4 / 3 (+0..2)` phones **every single day** purely as a constant flow, completely independent of how many phones they already own or how good yesterday's phone was!

- **Why an Economist Flags This:** Smartphones are **durable goods**, not daily meals. When a cohort buys a high-reliability phone with a 500-cycle battery and long OS support, near-term demand **saturates** until physical degradation or a major technological leap (ΔU) justifies replacement. Conversely, shipping fragile, overheating phones creates short-term warranty losses (`RMA`) and destroys **Brand Reputation Capital**, punishing corner-cutting in future launches.

10. Perfect Consumer Omniscience & Zero Working-Capital Timing Risk

- **What the code does:** The millisecond `d.released = true`, every consumer knows the exact physical `evaluation` score with zero uncertainty, and brands only buy production parts *after* validation succeeds on spot cash.
- **Why a Hardware CEO Flags This:** In real electronics:
    1. Brands must commit working capital (deposits/POs) **before** launch (`Working-Capital Valley of Death`); a validation delay creates genuine cash-flow peril.
    2. Smartphones are **experience goods**: initial launch velocity depends partly on **Brand Trust** and positioning, while post-launch reviews and field failures reveal true sustained thermal/battery quality over subsequent days/weeks.

Part II — How to Fix These Flaws Properly (Economically Sound Remedies)

All remedies below preserve **100% integer satang conservation**, **deterministic execution**, **immutable `human-core.js`**, and **observer-only Thai UI**:

Fix 1: Close the Macroeconomic Stock-Flow Loop (`SFC Velocity Closure`)

- **Household Income Recycling:** Each day, employed workers transfer discretionary spending above a prudence-scaled personal savings target (`person.cash > personalReserve`) into `w.households` / `w.segments` via `transfer()`.
- **Upstream Factor Recycling & Renewable Upstream Extraction:** `w.treasury` represents the rest of the macroeconomy (miners, power grid, food producers, equipment vendors). Each day:
    1. `w.treasury` recycles a bounded share of its cash above reserve back into `w.segments` (representing wages paid to external-sector households who buy phones).
    2. `w.treasury` converts paid extraction revenue into newly processed upstream raw stock (`treasury.stock`) at a bounded daily extraction capacity (updating `materials.opening` / `materials.extracted` explicitly in `audit()`),

preventing arbitrary Year-3 raw-material extinction while keeping supply finite per day.

Fix 2: Replace `candidates[0]` with Deterministic Multinomial Logit Choice

- Replace winner-take-all sorting in `settleMarket()` with **Logit Choice Probabilities** (using the simulation's deterministic seeded hash PRNG): $$P(\text{choose } j \mid \text{segment } s) = \frac{\exp(\beta_s \cdot U_{j,s})}{\exp(\beta_s \cdot U_{\text{outside}, s}) + \sum_{k \in \mathcal{A}} \exp(\beta_s \cdot U_{k,s})}$$
  - Include an **Outside Option ($U_{\text{outside}, s}$)**: if all available phones are overpriced or poor quality, buyers **keep their cash and defer purchase** rather than buying mediocre phones just because `utility >= 0.2`!
  - Result: Better value-for-money phones win the *largest* share (e.g., 55%), second-best wins ~30%, niche rival wins ~15%, and alphabetical ID bias (`'d1'` vs `'d2'`) disappears completely.

Fix 3: Scale-Consistent Nondimensionalization (Calibrate Wages, ASPs, BOM, and NRE Together)

Calibrate all monetary constants around a coherent **Simulated Thai SME Electronics Scale** (where 1 simulated firm = 3–12 key staff representing an operating unit, and total market = ~5,000–12,000 units/year):

Economic Variable	Current Broken Value	Recommended Calibrated Scale	Economic Rationale
Engineer / Specialist Daily Wage	THB 140–210	**THB 800 – 2,200/day** (role & skill-indexed)	Reflects skilled technical labor; poaching & headcount carry real burn-rate weight
CEO / Executive Daily Draw	THB 140–210	**THB 1,800 – 3,500/day**	Distinguishes executive overhead from junior staff
Unemployed Subsistence Buffer	THB 1,000 (~16 days)	**THB 15,000** + reservation wage decay after 30d unemployed	Prevents absurd Day-17 starvation while allowing realistic labor-market clearing

Economic Variable	Current Broken Value	Recommended Calibrated Scale	Economic Rationale
Segment Price Ceilings (`Value / Photo / Perf`)	THB 1,900 / 3,300 / 5,200	**THB 2,900 / 6,900 / 15,900** (or era-indexed)	Restores realistic gross-margin differentiation between commodity feature phones and high-R&D flagships
Custom Chip NRE + Mask Cost	THB 400 (tiny) or $80M (impossible)	**THB 450,000 – 2,500,000** (node-scaled)	Sets $Q_{\text{breakeven}}$ at **~400–1,200 units** (~10%–25% of annual market)—a true high-stakes strategic gamble!

Fix 4: Competitive Elasticity-Aware Pricing & Sunk-Cost Clearance Liquidation

- **Normal Pricing:** Anchor target price between marginal replacement cost and **competitor utility-adjusted benchmark price** in the target segment, not blind cost-plus.
- **Distress / Obsolete Clearance:** Drop the rigid `cost * 1.08` floor when **(a)** a newer generation has launched (`d.isLegacy`), or **(b)** `firm.cash < 14 * dailyWageBill`. Allow markdowns down to `0.55 * estimatedCost` (or market-clearing price) so cash-strapped firms rationally liquidate old inventory to survive!

Fix 5: Demand-Driven Supplier Production & Dynamic Inventory Quotes

- **Order-Signal Production:** Suppliers allocate WIP toward SKUs/generations that have **active downstream demand, unfilled RFQs, or low days-of-inventory**, halting production of obsolete `level 0` stock once brand demand shifts upward (except small service/replacement buffers).
- **Dynamic Quote Adjustment:** Adjust supplier `activePriceBps` weekly within `[7000, 14500]`:
  - Excess inventory + zero recent sales $\rightarrow$ cut quote price by `3%–5%` toward marginal raw+labor cost.
  - High utilization / low stock cover $\rightarrow$ raise quote price by `4%–8%` (scarcity rent).
- **Open EMS Competition:** Replace hardcoded `firm(w, 'supplier-ems')` with `supplierQuotes()` over all qualified `ems` / `osat_packaging` firms, and introduce

batch size tiers (`12 / 36 / 96` units) with setup-cost amortization (real economies of scale).

Fix 6: Insolvency Resolution, Restructuring & Zombie-Firm Prevention

- If a firm misses payroll (`unpaidDays >= 7`):
  1. **Stage 1 (Liquidity Defense):** Freeze new NRE/R&D, pause non-essential procurement, and trigger clearance markdowns on finished goods.
  2. **Stage 2 (Employee Attrition):** Remove the `people.length > 3` poaching immunity for firms in wage default (`unpaidDays >= 10`), and allow unpaid employees (`unpaidDays >= 14`) to voluntarily resign into the unemployed pool (`employerId = null`).
  3. **Stage 3 (Distress Asset Auction / Pivot):** Allow insolvent firms to sell excess raw/component lots back to solvent peers at a wholesale discount (`70%` of market quote) via `transfer()`, recycling trapped capital and physical parts back into the living industry.

Fix 7: Role-Weighted Commercial Pipeline & Durable-Good Installed Base

- **Decaying Commercial Readiness (`salesPipeline`):** Replace midnight reset `f.salesWork = 0` with `f.salesWork = f.salesWork * 0.80` (20% daily decay, capped by team commercial capacity), allowing pre-launch preparation for ~1–2 weeks before launch without hoarding infinite sales points, and weight contribution by each employee's `sales` subject rather than taxing pure lab engineers flatly during early R&D.
- **Durable Installed Base + Warranty/Brand Trust:** Implement cohort `installedBase` buckets (`v0.4.0`), warranty RMA cash/part liabilities, and `firm.brandTrust` (`0..100`) updated by field reliability vs. launch positioning—turning quality engineering from a "nice spreadsheet stat" into a hard financial survival mechanism.

Part III — Prioritized Executive Action Matrix for AI Developers

Priority	Economic / Business Issue	Files Affected	Complexity	Impact on Realism & Fun
P0 (Critical)	**Fix 2:** Replace `candidates[0]` + ID tie-break with seeded Multinomial Logit + Outside Option	`SPINOFF_SOURCE/js/sim.js`	Low (~35 LOC)	Eliminates fake 100%-vs-0% monopolies immediately

Priority	Economic / Business Issue	Files Affected	Complexity	Impact on Realism & Fun
P0 (Critical)	**Fix 1:** Stock-Flow Consistent household/treasury cash velocity + bounded upstream raw extraction	`SPINOFF_SOURCE/js/sim.js`	Medium	Prevents Year 2–3 deflationary collapse while keeping `audit()` exact
P0 (Critical)	**Fix 4 & 5:** Remove `cost * 1.08` sunk-cost floor on clearance; stop blind `level 0` supplier overproduction; fix hardcoded `supplier-ems`	`SPINOFF_SOURCE/js/sim.js`	Low–Medium	Stops irrational dead-stock hoarding & makes EMS startups playable
P1 (High)	**Fix 3:** Scale-consistent calibration of wages, segment ASP ceilings, and custom R&D NRE ($Q_{\text{breakeven}}$ = 10–25% of annual TAM)	`sim.js`, `phone-engineering.js` (via options, respecting locks)	Medium	Makes Make-or-Buy a genuine high-stakes boardroom dilemma
P1 (High)	**Fix 6 & 8:** Zombie-firm distress liquidation + unpaid resignation + decaying	`SPINOFF_SOURCE/js/sim.js`, `ui.js`	Medium	Creates real bankruptcies, turnarounds, and talent reallocation

Priority	Economic / Business Issue	Files Affected	Complexity	Impact on Realism & Fun
	`salesWork` pipeline			
P2 (Structural)	**Fix 7:** Durable-good `installedBase`, thermal/battery RMA warranty costs, `brandTrust`, and batch MOQ/EOS	`SPINOFF_SOURCE/js/sim.js`, `ui.js`	Medium–High	Connects physical engineering (`surfaceC`, `cycleLife`) directly to long-term P&L

Summary Takeaway for the Owner & Development Team

Right now, *Silicon & Handset* has **world-class accounting discipline** (`audit()` conservation) paired with **naive microeconomic decision rules** (cost-plus floors, deterministic winner-take-all demand, frozen supplier quotes, uncalibrated wages/ASPs, and non-circulating household cash).

Upgrading those behavioral and market-clearing equations—without breaking a single rule in `AGENTS.md`—will transform the simulation from a *deterministic tech-level race* into a **genuinely emergent industrial economy** where niche component specialists, disciplined value brands, and ambitious custom-silicon gamblers can each thrive—or go spectacularly bankrupt—for completely logical, observable business reasons.